

Ranking Mathematical Candidates

A Structural Selection Layer for Discovery and Counterexample Search

Christof Krieg

MAAT Project – Independent Research

<https://maat-research.com>

2026

Abstract

Mathematical discovery often fails before proof begins: the right candidate construction is never selected for serious proof effort. This note proposes a MAAT-inspired structural-selection layer for ranking mathematical candidates before proof and auditing them after proof. Candidate constructions, counterexamples, and proof programs are scored by logical coherence, constraint balance, constructive novelty, cross-domain connectivity, and verification robustness.

The 2026 OpenAI unit-distance result motivates the framework. It shows that a central discrete-geometry belief can be overturned by a construction whose decisive bridge comes from algebraic number theory. The lesson is methodological: discovery systems need mechanisms that keep unusual but structurally coherent bridge candidates alive long enough to be tested.

The proposed pipeline is

generate broadly $\rightarrow$ select structurally $\rightarrow$ prove rigorously $\rightarrow$ verify independently.

MAAT supplies the selection and audit layer. Certification is still supplied by ordinary mathematics: checkable proof, independent scrutiny, and, where possible, formal verification.

Disclaimer

This paper does not reproduce, verify, or extend the OpenAI unit-distance proof. It uses that event only as motivation for a general methodology: structural ranking of mathematical discovery candidates. MAAT is used here as a search-prior and robustness-audit layer; proof remains the external certification step.

1 Motivation

In May 2026, OpenAI reported that an internal reasoning model disproved a long-standing conjectural belief in the planar unit distance problem: the expectation that the maximal number of unit-distance pairs among n points should be $n^{1+o(1)}$. The announced result gives infinitely many configurations with at least $n^{1+\delta}$ unit distances for some fixed $\delta > 0$, using ideas from algebraic number theory rather than only the traditional geometric constructions.

The important lesson for this paper is methodological. The discovery was not merely a longer search inside the most familiar family of square-grid constructions. It depended on a bridge:

discrete geometry $\longleftrightarrow$ algebraic number theory.

This suggests a general question:

Can a structural-selection framework help identify which mathematical candidates are worth trying to prove?

The answer proposed here is operational. MAAT can help as a ranking and filtering layer that moves promising candidates into the proof pipeline. Certification remains the job of proof and verification.

2 Mathematical Discovery as Structural Selection

Let $\mathcal{C}$ be a space of candidate mathematical objects: point configurations, graphs, number fields, inequalities, proof strategies, SAT encodings, extremal examples, or proposed counterexamples. Let $\mathcal{P}$ be a space of proof states or proof programs.

A mathematical discovery process can be decomposed as

$$\mathcal{C} \xrightarrow{\mathcal{R}_{\text{rank}}} \mathcal{C}_{\text{promising}} \xrightarrow{\mathcal{P}} \text{proof attempts} \xrightarrow{\mathcal{V}_{\text{proof}}} \text{accepted or rejected results.}$$

The structural-selection layer acts primarily on the first arrow:

$$\mathcal{R}_{\text{rank}} : \mathcal{C} \rightarrow \mathbb{R}.$$

Its goal is to spend scarce reasoning time on candidates whose structure is more likely to survive proof-level scrutiny before the verifier $\mathcal{V}_{\text{proof}}$ is invoked.

3 The Discovery Supports

For mathematical discovery, the four primary MAAT supports and the emergent robustness closure can be interpreted as follows.

Support	Name	Discovery interpretation
H	Harmony / coherence	Internal logical consistency; lemmas fit together; no hidden contradiction or impossible assumption.
B	Balance	Constraint compatibility; the candidate satisfies competing requirements without solving one by destroying another.
S	Activity / construction	Nontrivial generative power; the candidate creates new examples, parameters, or degrees of freedom.
V	Connectivity	Bridges between distant mathematical domains, techniques, or invariants.
R_{rob}	Robustness	Stability under perturbation, parameter change, independent checking, and formalisation pressure.

The central point is that V and S should not be decorative. In major discoveries, the useful candidate may look unlikely precisely because it lives outside the dominant local search mode of the field.

4 A Candidate Ranking Functional

For each candidate $X \in \mathcal{C}$, define nonnegative defect scores

$$d_a[X] \geq 0, \quad a \in \{H, B, S, V\}.$$

The corresponding supports are

$$\Gamma_a[X] = \frac{1}{1 + d_a[X]}. \quad (1)$$

A simple structural discovery functional is

$$F_{\text{MAAT}}^{\text{disc}}[X] = - \sum_{a \in \{H, B, S, V\}} \lambda_a \log(\varepsilon + \Gamma_a[X]) - \lambda_{\text{cl}} \log(\varepsilon + R_{\text{rob}}[X]). \quad (2)$$

Lower values indicate higher structural support for the proof queue. The robustness closure may use the standard v1.2.1 form

$$R_{\text{resp}} = (HBV)^{1/3}, \quad R_{\text{rob}} = \min(R_{\text{resp}}, (HBSV)^{1/4}). \quad (3)$$

5 Counterexample Search

Counterexample search is a natural setting for structural selection. Given a conjecture

$$\forall X \in \mathcal{C} : P(X),$$

the goal is to find X such that $\neg P(X)$. Naive search often stays near known examples. Structural search should instead promote candidates that satisfy existing constraints while activating unusual bridge structure.

For a counterexample candidate X , useful defects include:

$$d_H[X] = \text{internal inconsistency or failed lemma count}, \quad (4)$$

$$d_B[X] = \text{constraint imbalance or unsatisfied side conditions}, \quad (5)$$

$$d_S[X] = \text{lack of generative degrees of freedom}, \quad (6)$$

$$d_V[X] = \text{absence of cross-domain mechanisms}, \quad (7)$$

$$d_R[X] = \text{instability under perturbation or verification}. \quad (8)$$

The discovery process then becomes:

generate candidates $\rightarrow$ rank by $F_{\text{MAAT}}^{\text{disc}}$ $\rightarrow$ attempt proof $\rightarrow$ audit robustness.

6 Bridge Detection

The OpenAI unit-distance case highlights the importance of bridge detection. The relevant discovery event was not simply that a candidate was complex or novel. It was that an unexpected translation preserved enough structure to make a geometric consequence accessible through number-theoretic machinery.

The following bridge score should be read as a first operational prototype, not as a canonical definition. Its purpose is to make the connectivity sector measurable enough to compare candidate families and null models.

Define a bridge graph whose nodes are mathematical domains, objects, and techniques, and whose edges represent usable translations:

$$\mathcal{B} = (\mathcal{D}, \mathcal{E}).$$

For a candidate X , let $\mathcal{B}_X$ be the subgraph of domains and methods actively used by the construction. Bridge quality should distinguish genuine structural transfer from mere analogy. One useful decomposition is:

Quantity	Meaning
b_{reach}	How far the candidate moves from the dominant local search family.
b_{pres}	How much structure is preserved under translation.
b_{yield}	Whether the bridge creates new constructions, bounds, or invariants.
b_{cost}	Translation defects, hidden assumptions, or loss of checkability.

In a reproducible benchmark these quantities should be instantiated with predeclared proxies. For example, b_{reach} may count distance from known construction families, b_{pres} may count preserved constraints or invariants, b_{yield} may measure the number of new parameter families or bound improvements produced, and b_{cost} may count auxiliary assumptions, translation failures, or unverified lemmas. The exact proxies must be domain-specific, but the decomposition forces the test to distinguish useful bridges from merely exotic analogies. In the extremal-graph toy setting below, these proxies can be made concrete: b_{reach} can be the graph-edit or feature-vector distance from standard families, using degree histograms, edge density, triangle density, small-cycle counts, or spectral moments; b_{pres} can be the fraction of sampled candidates preserving the forbidden-substructure constraint; b_{yield} can be the number of parameter values improving the current edge baseline; and b_{cost} can count extra unproved assumptions, exceptional cases, or failed construction instances.

A bridge support can then be written as

$$V[X] = \frac{b_{\text{reach}}(X) b_{\text{pres}}(X) b_{\text{yield}}(X)}{1 + b_{\text{cost}}(X)}. \quad (9)$$

This makes explicit why some surprising ideas are valuable and others are not. A bridge is not useful merely because it is surprising. It is useful when the translation preserves enough structure to produce checkable consequences in the target problem.

For counterexample search, bridge detection changes the search policy. Instead of asking only

Which known family can be pushed harder?

one also asks

Which external structure preserves the conjecture’s constraints while changing the growth mechanism?

This is where MAAT’s connectivity sector becomes operational rather than decorative. It rewards bridges that are distant enough to escape local bias, but coherent enough to survive verification.

7 Toy Example: Ranking Extremal Graph Candidates

As a minimal illustration, consider a toy extremal-graph task:

Find graphs on n vertices with many edges while avoiding a forbidden substructure F .

This is not meant as a new theorem. It is a small benchmark template for making the ranking idea concrete.

Let X be a candidate graph family. For a very small numerical benchmark, one could enumerate labelled or isomorphism-reduced candidates for $n \leq 10$, enumerate restricted construction families up to $n \leq 12$, and sample larger cases up to about $n \leq 20$. The defects can then be computed directly by subgraph counting, edge-count comparison, graph-feature distance to known families, exhaustive single-edge sampling for very small n , and random edge-sampling for larger sampled cases. One possible predeclared defect map is:

$$d_H[X] = \frac{\text{\#violations of the forbidden condition}}{1 + \#E(X)}, \quad (10)$$

$$d_B[X] = \left| \frac{\#E(X)}{\#E_{\text{target}}} - 1 \right| + \frac{\text{\#side-condition failures}}{1 + n}, \quad (11)$$

$$d_S[X] = \frac{1}{1 + \text{\#free construction parameters}}, \quad (12)$$

$$d_V[X] = \frac{1}{1 + b_{\text{reach}} b_{\text{pres}} b_{\text{yield}}}, \quad (13)$$

$$d_R[X] = \text{instability under small parameter changes.} \quad (14)$$

Here d_S rewards constructions that generate a family rather than a single isolated example, while d_V rewards bridges to external mechanisms such as finite fields, algebraic constructions, coding-theoretic objects, spectral methods, or number-theoretic parameterisations.

The benchmark would compare three search policies:

Policy	Meaning
Known-family search	Mutate standard examples only.
Random search	Sample candidates without structural ranking.
MAAT-ranked search	Generate broadly, then prioritise low $F_{\text{MAAT}}^{\text{disc}}$ candidates.

The success criterion is not whether MAAT proves the extremal statement. It is whether the MAAT-ranked policy finds proof-worthy candidates or genuine violations faster than known-family and random baselines under the same compute budget. This toy setup also makes bridge detection measurable: b_{reach} is graph-feature distance from known families, b_{pres} is the fraction of forbidden-substructure constraints preserved, b_{yield} is the number of parameter values that improve the current edge baseline, and b_{cost} counts extra assumptions, failed translations, or unverified construction cases.

8 Generator Stationarity for Proof Search

The v1.5 generator-stationarity lesson applies directly to mathematical discovery. Proof search has a generator:

$$\mathcal{G}_{\text{proof}} : \text{proof state} \rightarrow \text{next proof state}.$$

This generator may be a human proof strategy, a theorem prover tactic, an LLM reasoning trajectory, a SAT/SMT solver, or a formal verification pipeline.

A proof candidate is structurally stronger if its key invariants survive this generator. Thus one can define a stationarity defect

$$d_{\text{stat}}[X] = \frac{\|\mathcal{G}_{\text{proof}} I_X - I_X\|}{\Xi + \|I_X\|}, \quad (15)$$

where I_X denotes the proposed invariant, lemma structure, or bridge translation carried by X .

This turns v1.5 into a discovery criterion. A bridge or invariant should not only look promising at the start of a proof attempt; it should remain stable under the proof moves that try to use

it. In this sense, proof search is an evolving system: tactics, reductions, normal forms, and auxiliary lemmas act like generator steps. A candidate with high generator-stationarity keeps its central invariant usable as these steps accumulate; a weak candidate loses the invariant as soon as the proof is forced to become precise. In the extremal-graph toy example, this would test whether the key invariant “many edges while avoiding F ” survives edge switches, parameter shifts, compression steps, or reductions to smaller witness graphs. Numerically, one could apply all single-edge additions/deletions for $n \leq 10$, recompute the number of forbidden subgraphs, and measure how often the edge advantage and F -avoidance invariant survive. This gives a practical test:

Does the candidate remain coherent as the proof attempt evolves, or does the proof generator immediately destroy its key invariant?

9 A Practical Protocol

A MAAT-style mathematical discovery workflow should follow a strict protocol.

1. Define the conjecture, candidate space, and allowed construction rules.
2. Define defects d_H, d_B, d_S, d_V before looking at final successes.
3. Generate candidates using diverse local and cross-domain mechanisms.
4. Rank candidates by $F_{\text{MAAT}}^{\text{disc}}$.
5. Attempt proof only on the top-ranked and selected control candidates.
6. Compare against baselines: random search, energy-size heuristics, known-family search, and expert-prior ranking.
7. Apply robustness audits: parameter variation, independent checking, formalisation pressure, and adversarial lemma review.
8. Report failures and false positives.

The last step is essential. A discovery ranking framework is only scientifically meaningful if it can fail.

10 Capabilities and Boundaries

Discovery function	Boundary condition
Prioritise promising constructions	They still require rigorous proof.
Detect cross-domain bridges	The bridge must be checked by domain experts.
Keep long-shot candidates alive	Novelty alone is not evidence.
Compare candidate families	Baselines and controls are required.
Audit robustness after proof	Verification remains external.

The operational slogan is therefore:

MAAT does not prove the theorem; it ranks which mathematical structures are worth trying to prove.

11 Limitations

- The proposed defects are methodological and domain-dependent.
- The framework has not been tested on the OpenAI unit-distance proof.
- High novelty can produce many false positives.
- Bridge detection is difficult to formalise without expert input.
- Verification robustness must remain external to the ranking score.
- The framework should be evaluated against genuine mathematical benchmarks before any universality claim is made.

12 Reproducibility and Status

Repository:

<https://github.com/Chris4081/structural-selection-principle>

This is a conceptual methodology paper. It introduces no new dataset and does not redistribute the OpenAI proof or companion files.

Source and attribution note. The OpenAI unit-distance article is used only as a cited public source and motivating example. No OpenAI figures, proof PDFs, companion files, model outputs, or extended article text are reproduced in this paper. No endorsement by OpenAI or by the mathematicians quoted in the OpenAI article is implied.

A future reproducible benchmark should implement candidate ranking on toy extremal-graph, SAT, or discrete-geometry construction tasks with predefined defects and shuffled-structure nulls.

13 Conclusion

The OpenAI unit-distance result is important not only because it settles a long-standing mathematical belief, but because it illustrates the value of unexpected structural bridges. The lesson is direct: mathematical discovery can be treated as structural selection over candidate constructions and proof programs.

The role of MAAT is to rank coherence, balance, construction power, cross-domain connectivity, and robustness before expensive proof effort is spent. The proposed discovery layer is therefore simple:

generate broadly $\rightarrow$ select structurally $\rightarrow$ prove rigorously $\rightarrow$ verify independently.

Its value is clearest in counterexample search, where the decisive candidate may lie outside the dominant local construction family.

References

- [1] OpenAI, “An OpenAI model has disproved a central conjecture in discrete geometry,” OpenAI Research, May 20, 2026. <https://openai.com/index/model-disproves-discrete-geometry-conjecture/>.
- [2] P. Erdős, “On sets of distances of n points,” *The American Mathematical Monthly* **53**, 248–250 (1946).

- [3] J. Spencer, E. Szemerédi, and W. T. Trotter, “Unit distances in the Euclidean plane,” in *Graph Theory and Combinatorics*, Academic Press, 293–303 (1984).
- [4] G. Pólya, *Mathematics and Plausible Reasoning*, Princeton University Press (1954).
- [5] I. Lakatos, *Proofs and Refutations: The Logic of Mathematical Discovery*, Cambridge University Press (1976).
- [6] E. T. Jaynes, “Information Theory and Statistical Mechanics,” *Physical Review* **106**, 620–630 (1957).
- [7] C. Krieg, “Generator Stationarity and Dynamical Defect-Field Geometry: Refining Structural Selection for Evolving Systems,” preprint (2026).